

## Лабораторная работа №7. Flutter: иконка, сборка и публикация

**Цель работы:** доделать приложение слот-машины. Добавить воспроизведение звука. Создать иконку приложения в Krita, подключить её к Flutter-проекту, собрать финальный билд под Web и Android, оформить репозиторий на GitHub как полноценный публичный проект.

### Часть 1. Подготовка — выбираем проект

Для финальной лабораторной возьмите предыдущее **приложение** со слот-машиной.

1. Создайте его копию.
2. Откройте проект в VS Code.
3. Убедитесь что приложение запускается:

**flutter run -d chrome**

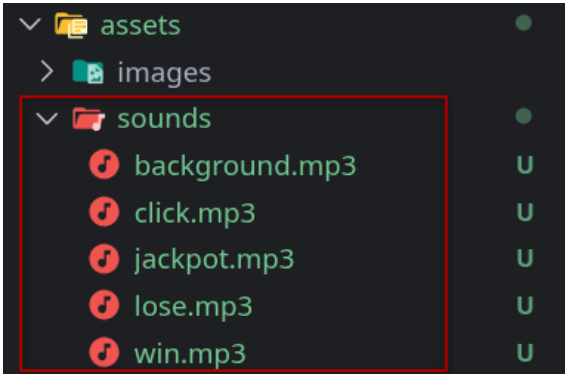
### Часть 2. Звук для слот-машины (Web Audio API)

Добавим звуки без единого пакета с pub.dev — через встроенный Web Audio API браузера напрямую из Dart.

⚠ Эта версия звука работает только в браузере. В Части 5 мы заменим её на кроссплатформенное решение. Пока изучаем принцип работы.

### Подготовка аудиофайлов

Скачайте архив со звуками с сайта и поместите файлы в папку **assets/sounds/**:



assets/sounds/

- background.mp3 ← фоновая музыка (зацикленная)
- win.mp3 ← звук победы
- lose.mp3 ← звук проигрыша
- jackpot.mp3 ← звук джекпота
- click.mp3 ← звук нажатия кнопки

Зарегистрируйте папку в **pubspec.yaml**:

```
flutter:  
  uses-material-design: true  
  assets:  
    - assets/images/cherry.png  
    - assets/images/lemon.png  
    - assets/images/seven.png  
    + - assets/sounds/
```

Выполните в терминале:

**flutter pub get**

## Шаг 1. Создаём сервис звука

В этом шаге мы создадим **отдельный сервис для работы со звуком**. Он будет отвечать за:

- воспроизведение фоновой музыки;
- воспроизведение коротких звуковых эффектов (победа, проигрыш, нажатие кнопки);
- управление отключением и включением звука.

Такой подход позволяет **вынести всю работу со звуком в один файл**, чтобы не дублировать код в разных частях приложения.

Этот вариант — только для первого знакомства с **Web Audio API**. В Части 5 мы заменим его на кроссплатформенный пакет **audioplayers**, который работает и на Web, и на Android.

1. Создайте файл **lib/sound\_service.dart**

2. Откройте его и начнём писать код.

### Пишем код

#### 1. Добавляем импорт

```
// ignore: deprecated_member_use, avoid_web_libraries_in_flutter
import 'dart:html' as html;
```

Здесь мы импортируем библиотеку **dart:html**, которая позволяет работать с HTML-элементами браузера.

В частности, мы будем использовать класс **AudioElement**, который позволяет:

- загружать аудиофайлы;
- управлять громкостью;
- запускать и останавливать воспроизведение.

Этот способ подходит для **Flutter Web**, потому что звук воспроизводится через возможности браузера.

#### 2. Создаём класс сервиса

```
class SoundService { }
```

Мы создаём отдельный класс, который будет выполнять роль **сервиса управления звуком**.

Все методы этого класса будут отвечать за различные операции:

- запуск фоновой музыки
- воспроизведение эффектов
- включение и выключение звука

### 3. Добавляем поля класса

```
class SoundService {  
    static html.AudioElement? _backgroundMusic;  
    static bool _isMuted = false;  
    +static bool get isMuted => _isMuted;  
}
```

Здесь объявляются переменные:

- **\_backgroundMusic** — объект фоновой музыки
- **\_isMuted** — флаг, показывающий, выключен ли звук
- **isMuted** — публичный геттер, позволяющий проверить состояние звука

Обратите внимание, что переменные объявлены как **static**.

Это означает, что:

- они принадлежат **самому классу**, а не конкретному объекту;
- их можно использовать **без создания экземпляра класса**.

Таким образом, класс используется как **глобальный сервис**, доступный из любой части приложения.

### 4. Запуск фоновой музыки

```
static void playBackground() {  
    _backgroundMusic =  
        html.AudioElement('assets/assets/sounds/background.mp3')  
        ..loop = true  
        ..volume = 0.4;  
    if (!_isMuted) {  
        _backgroundMusic!.play();  
    }  
}
```

1. Создает объект **AudioElement**

2. Загружает файл фоновой музыки

3. Устанавливает параметры:

- **loop = true** — музыка будет играть по кругу

- **volume = 0.4** — уровень громкости

4. Если звук не отключён, запускает воспроизведение.

Фоновая музыка хранится в переменной `_backgroundMusic`, чтобы позже её можно было **остановить или возобновить**.

💡 **Почему assets/assets/...?** Flutter при web-сборке помещает все assets в папку assets/ внутри build/web/. Поэтому путь для браузерного Audio API удваивается: первый assets/ — папка в build, второй — ваша папка в проекте. Это особенность только Web-платформы.

## 5. Воспроизведение одnorазовых звуков

```
static void _playSound(String path, {double volume = 1.0}) {  
  if (_isMuted) return;  
  html.AudioElement(path)  
    ..volume = volume  
    ..play();  
}
```

Этот метод используется для **воспроизведения коротких звуковых эффектов**.

Он принимает:

- **path** — путь к звуковому файлу
- **volume** — уровень громкости (по умолчанию 1.0)

Если звук отключён (`_isMuted == true`), метод просто завершает работу.

Метод объявлен как **приватный** (`_playSound`), потому что он используется только внутри этого класса.

## 6. Методы для различных звуков

```
static void playWin() => _playSound('assets/assets/sounds/win.mp3');  
static void playJackpot() =>  
  _playSound('assets/assets/sounds/jackpot.mp3');  
static void playLose() =>  
  _playSound('assets/assets/sounds/lose.mp3', volume: 0.7);  
static void playClick() =>  
  _playSound('assets/assets/sounds/click.mp3', volume: 0.6);
```

Здесь создаются **удобные методы для разных событий игры**:

- звук победы
- звук джекпота
- звук проигрыша
- звук нажатия кнопки

Такой подход похож на использование **Singleton-сервиса**, поскольку класс используется как **единственная точка управления звуком** во всём приложении.

## 7. Включение и выключение звука

```
static void toggleMute() {  
  _isMuted = !_isMuted;  
  if (_isMuted) { _backgroundMusic?.pause(); }  
  else { _backgroundMusic?.play(); }  
}
```

Этот метод переключает состояние звука:

- если звук был включён — он отключается;
- если был отключён — включается.

При отключении звука:

- фоновая музыка **ставится на паузу**

При включении:

- музыка **возобновляет воспроизведение**.

## Шаг 2. Подключаем звук в SlotMachine

Откройте **lib/slot\_machine.dart** и добавьте импорт:

```
import 'sound_service.dart';
```

**Запуск фоновой музыки** — добавьте **initState()** в **\_SlotMachineState**. Этот метод вызывается один раз при создании виджета — идеальное место для инициализации:

```
@override  
void initState() {  
  super.initState();  
  SoundService.playBackground();  
}
```

**Звук нажатия кнопки** — добавьте в начало метода **\_spin()**:

```
Future<void> _spin() async {  
  if (_coins <= 0 || _isSpinning) return;  
  SoundService.playClick(); // ← добавьте эту строку  
  setState(() {
```

**Звуки результата** — обновите блок определения результата в **\_spin()**:

```

setState(() {
  _isSpinning = false;
  if (result1 == result2 && result2 == result3) {
    if (result1 == 'assets/images/seven.png') {
      _coins += 10;
      _message = 'ДЖЕКПОТ! 🎰🎰🎰 +10 монет';
      SoundService.playJackpot(); // ← джекпот
    } else {
      _coins += 3;
      _message = 'Победа! 🎉 +3 монеты';
      SoundService.playWin(); // ← победа
    }
  } else {
    _coins -= 1;
    _message = 'Попробуй ещё раз 😞 -1 монета';
    SoundService.playLose(); // ← проигрыш
  }
});

```

### Шаг 3. Кнопка включения/выключения звука

Добавьте кнопку **mute** в **build()** в **\_SlotMachineState**. Поскольку состояние кнопки (**\_isMuted**) хранится в **SoundService**, нам нужна локальная переменная для перерисовки:

Добавьте переменную состояния внутрь класса:

```

var _isMuted = false;

```

Добавьте метод **\_toggleMute()** перед **build()**:

```

void _toggleMute() {
  SoundService.toggleMute();
  setState(() {
    _isMuted = SoundService.isMuted;
  });
}

```

Добавьте кнопку в **build()** — например в самый верх **Column**, перед счётчиком монет:

```

return Column(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Align(
      alignment: Alignment.topRight,
      child: Padding(
        padding: EdgeInsets.only(right: 16, top: 8),
        child: IconButton(
          onPressed: _toggleMute,
          icon: Icon(
            _isMuted
              ? Icons.volume_off
              : Icons.volume_up,
            color: Colors.white,
            size: 28,
          ), // Icon
        ), // IconButton
      ), // Padding
    ), // Align
    Text(

```

### Важный момент — политика браузера

Современные браузеры **блокируют** **автовоспроизведение** звука до первого взаимодействия пользователя со страницей. Это означает что фоновая музыка не запустится сразу при загрузке — только после первого нажатия кнопки.

Это не баг вашего кода — это намеренное ограничение браузера для защиты от назойливой рекламы. Чтобы обойти это, можно запускать **playBackground()** не в **initState()**, а при первом нажатии кнопки «КРУТИТЬ». Добавьте переменную в класс:

```
var _backgroundStarted = false;
```

В **Future<void> \_spin() async** добавьте:

```

Future<void> _spin() async {
  if (_coins <= 0 || _isSpinning) return;
  SoundService.playClick();
  setState(() {
    _isSpinning = true;
    _message = '';
  });
  if (!_backgroundStarted) {
    SoundService.playBackground();
    + _backgroundStarted = true;
  }
}

```

## Итоговый коммит:

**git add .**

**git commit -m "Bonus: add sound effects via Web Audio API"**

**git push**

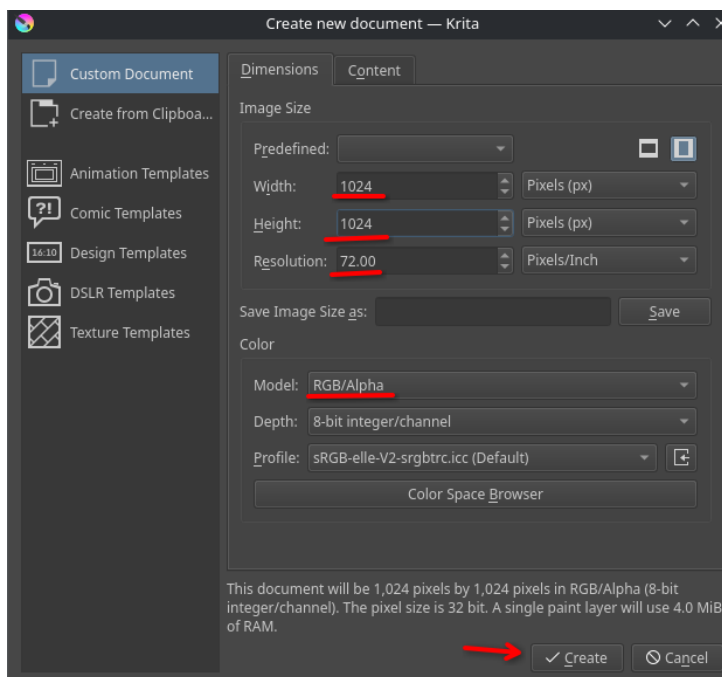
## Часть 3. Создаём иконку в Krita

### Шаг 1. Требования к иконке

Flutter ожидает иконку в виде квадратного PNG-файла. Требования:

| Параметр       | Значение                    |
|----------------|-----------------------------|
| Формат         | PNG                         |
| Размер         | 1024×1024 пикселей          |
| Фон            | Не прозрачный (для Android) |
| Название файла | app_icon.png                |

### Шаг 2. Рисуем иконку в Krita



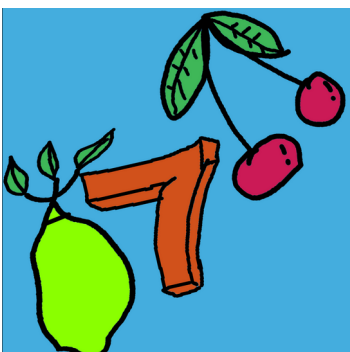
Откройте редактор Krita и создайте новый документ:

1. **File** → **New** (Ctrl+N)
2. Установите размер: **1024** × **1024** пикселей
3. Resolution: **72 ppi**
4. Color space: **RGB/Alpha, 8-bit**
5. Нажмите **Create**

Нарисуйте иконку для вашего приложения. Для слот-машины — например игровой автомат, кубики,

монеты или символы черешни/лимона/семёрки.

💡 Несколько советов для хорошей иконки:



- Используйте яркий однотонный фон — иконка должна читаться на любых обоях
- Центральный символ должен занимать ~60-70% площади
- Избегайте мелких деталей — иконка отображается маленькой на экране телефона



- Скруглите углы визуально — Android сам обрежет иконку по маске, но лучше заложить отступ

Экспортируйте готовую иконку:

1. **File** → **Export**
2. Выберите формат **PNG**
3. Сохраните как **app\_icon.png**

### Шаг 3. Подключаем иконку к проекту

Создайте папку **assets/icon/** в корне проекта и поместите туда **app\_icon.png**.

### Подключаем пакет **flutter\_launcher\_icons**

Это единственный внешний пакет в наших лабораторных — он генерирует иконки нужных размеров для всех платформ автоматически. Без него пришлось бы вручную создавать десятки файлов разных размеров.

Откройте **pubspec.yaml** и добавьте в раздел **dev\_dependencies**:

```
dev_dependencies:  
  flutter_test:  
    sdk: flutter  
  flutter_lints: ^6.0.0  
+ flutter_launcher_icons: ^0.14.3
```

И в **конец файла!!** добавьте конфигурацию иконки:

```
15 > dev_dependencies: ...  
20 > flutter: ...  
27  
28 > flutter_launcher_icons:  
29   android: true  
30   ios: false  
31 > web:  
32   generate: true  
33   image_path: "assets/icon/app_icon.png"  
34   image_path: "assets/icon/app_icon.png"  
35   min_sdk_android: 25  
36
```

⚠ Соблюдайте отступы — **flutter\_launcher\_icons**: должен быть на верхнем уровне файла, не внутри **flutter**

Сохраните файл и установите пакет:

**flutter pub get**

## Генерируем иконки

Введите в терминале:

**dart run flutter\_launcher\_icons**

Flutter сгенерирует все нужные размеры иконок для Android и Web и разложит их по нужным папкам автоматически.



**СКРИНШОТ 1:** Скриншот терминала с результатом генерации иконок.

Сохраните в **img/step1\_ВашаФамилия.png**

Выполните в терминале:

**git add .**

**git commit -m "Add app icon"**

## Часть 4. Сборка под Web

### Шаг 1. Что такое flutter build

До сих пор мы запускали приложение командой `flutter run` — это режим разработки с Hot Reload и отладчиком. Для публикации нужна **release-сборка**: оптимизированная, без инструментов отладки, значительно быстрее.

### Шаг 2. Собираем Web-версию

**flutter build web --release**

Сборка займёт 1–3 минуты. Flutter скомпилирует Dart в JavaScript и соберёт всё необходимое.

```
Compiling lib/main.dart for the Web...
```

```
20,0s
```

```
✓ Built build/web
```

Результат появится в папке **build/web/**:

**build/web/**

- |— **index.html**      ← точка входа
- |— **main.dart.js**   ← скомпилированный код
- |— **flutter.js**
- |— **icons/**          ← иконки приложения
- |— **assets/**        ← ваши картинки и ресурсы



Папку **build/web/** можно загрузить на **любой статический хостинг** — GitHub Pages, Netlify, Vercel — и приложение будет доступно по ссылке в интернете. Это полноценное веб-приложение.

 **СКРИНШОТ 2:** Скриншот папки **build/web/** в VS Code. Сохраните в **img/step2\_ВашаФамилия.png**

Выполните в терминале:

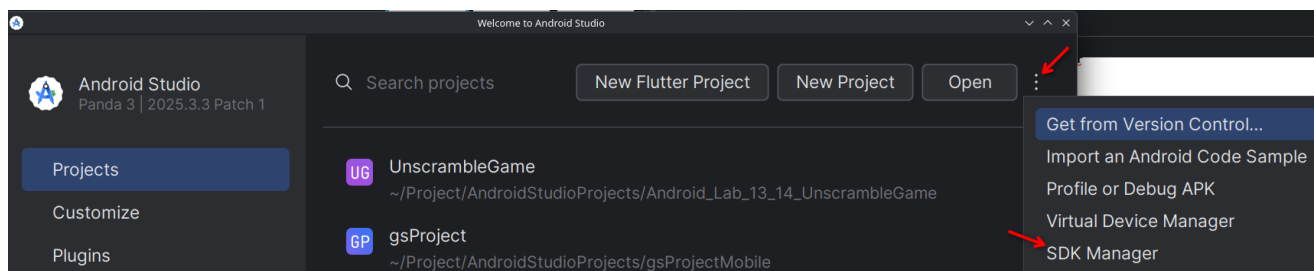
**git add .**

**git commit -m "Add web release build"**

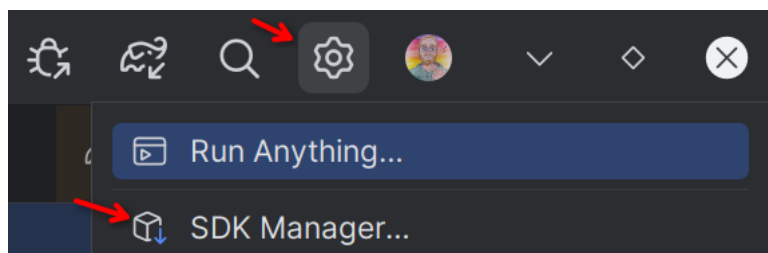
## Часть 5. Сборка под Android

### Шаг 0. Подготовка в Android Studio

Откройте Android Studio и выберите пункт SDK Manager. Если у вас открывается на главной вкладке, то нажмите на **три точки** — **SDK Manager**:

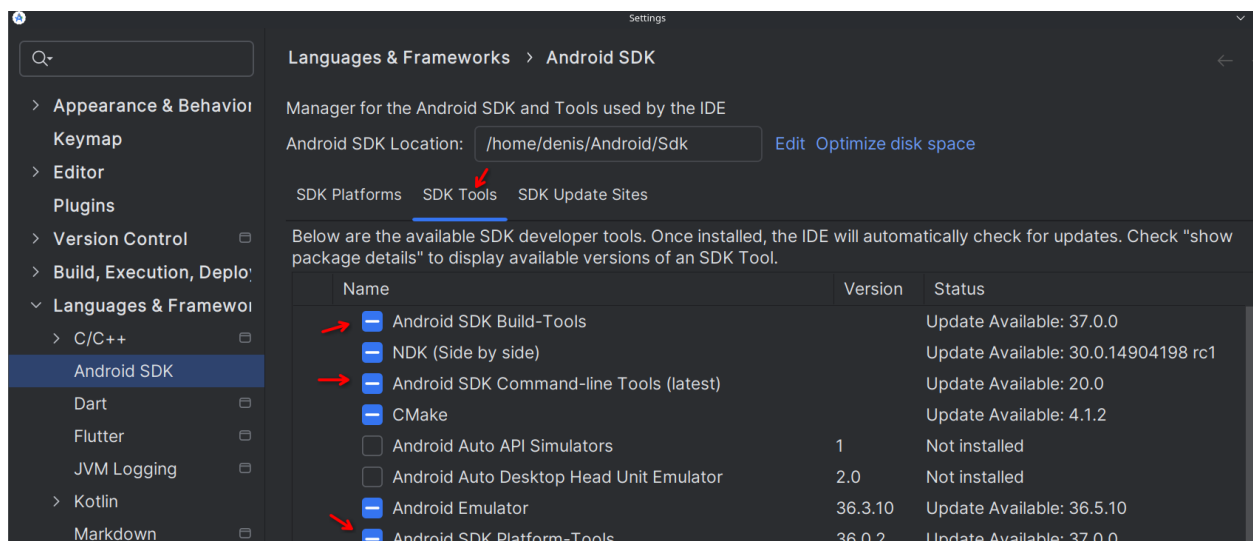


Если же у вас открылся проект, нажмите на **шестерёнку** — **SDK Manager**:



Проверьте что у вас установлены следующие инструменты:

- **SDK Build Tools**
- **SDK Command-line Tools**
- **SDK Platform-Tools**



## Шаг 1. Проверяем Android-окружение

### flutter doctor

Убедитесь что в выводе нет критических ошибок по Android SDK.

У вас выскочит предупреждение что нужно установить лицензию:

```
denis@DESKTOP-T3S701S MINGW64 ~
$ flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.38.9, on Microsoft windows [Version
    10.0.19045.5487], locale ru-RU)
[✓] Windows Version (Ъ @€а®6®дв windows 10 Pro 64-a §апх- п, 22H2, 2009)
[!] Android toolchain - develop for Android devices (Android SDK version 36.1.0)
    ↗ ! Some Android licenses not accepted. To resolve this, run: flutter doctor
    --android-licenses
```

Вводим предлагаемую команду:

### flutter doctor —andoird-licenses

Нажмите несколько раз **y**, для подтверждения:

```
denis@DESKTOP-T3S701S MINGW64 ~
$ flutter doctor --android-licenses
[=====] 100% Computing updates...
5 of 7 SDK package licenses not accepted.
Review licenses that have not been accepted (y/N)? ↗ y
```

## Шаг 2. Настройка названия приложения

Перед сборкой установим красивое название которое будет отображаться на телефоне. Откройте файл **android/app/src/main/AndroidManifest.xml** и найдите строку:

**android:label="slot\_machine"**

Замените на: **android:label="Слот-машина"**

### Обновление звука — audioplayers для Web и Android

Обновляем **SoundService** чтобы звук работал на обеих платформах через пакет audioplayers.

## Шаг 3. Добавляем пакет

Откройте **pubspec.yaml** и добавьте в раздел **dependencies**:

```
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  +audioplayers: ^6.0.0
```

Сохраните и установите через терминал:

flutter pub get

## Шаг 4. Полностью заменяем SoundService

Замените содержимое `lib/sound_service.dart`:

```
import 'package:audioplayers/audioplayers.dart';

class SoundService {
  static AudioPlayer? _backgroundPlayer;
  static bool _backgroundStarted = false;
  static bool _isMuted = false;

  // Пул плееров для звуковых эффектов
  static final List<AudioPlayer> _sfxPool = [];
  static int _sfxIndex = 0;
  static const int _poolSize = 4;

  static bool get isMuted => _isMuted;

  static Future<void> init() async {
    // Инициализируем пул плееров заранее
    for (int i = 0; i < _poolSize; i++) {
      final p = AudioPlayer();
      await p.setPlayerMode(
        PlayerMode.lowLatency,
      ); // быстрый режим для эффектов
      _sfxPool.add(p);
    }
  }

  static Future<void> playBackground() async {
    if (_backgroundStarted) return; // не запускать повторно
    _backgroundStarted = true;

    _backgroundPlayer = AudioPlayer();
    await _backgroundPlayer!.setReleaseMode(ReleaseMode.loop);
    await _backgroundPlayer!.setVolume(_isMuted ? 0.0 : 0.4);
    await _backgroundPlayer!.play(AssetSource('sounds/background.mp3'));
  }

  static Future<void> _playSound(String fileName, {double volume = 1.0}) async {
    if (_isMuted) return;

    // Берём плеер из пула по кругу
    final player = _sfxPool[_sfxIndex % _poolSize];
    _sfxIndex++;

    await player.stop();
    await player.setVolume(volume);
    await player.play(AssetSource('sounds/$fileName'));
  }
}
```

```

static Future<void> playWin() => _playSound('win.mp3');
static Future<void> playJackpot() => _playSound('jackpot.mp3');
static Future<void> playLose() => _playSound('lose.mp3', volume: 0.7);
static Future<void> playClick() => _playSound('click.mp3', volume: 0.6);

static Future<void> toggleMute() async {
  _isMuted = !_isMuted;
  if (_isMuted) {
    await _backgroundPlayer?.setVolume(0.0);
  } else {
    await _backgroundPlayer?.setVolume(0.4);
  }
}

static Future<void> dispose() async {
  await _backgroundPlayer?.dispose();
  for (final p in _sfxPool) {
    await p.dispose();
  }
  _sfxPool.clear();
  _backgroundStarted = false;
}
}

```

Обратите внимание на `AssetSource('sounds/background.mp3')` — путь указывается без `assets/`, `audioplayers` сам добавляет этот префикс.

### Шаг 5. Обновляем вызовы в `slot_machine.dart` и `main.dart`

Обновите `main.dart` чтобы инициализировать пул до запуска:

```

import 'package:flutter/material.dart';
import 'package:slot_machine/slot_machine.dart';
import 'package:slot_machine/sound_service.dart';

Run | Debug | Profile
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await SoundService.init();
  runApp(
    MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        backgroundColor: Colors.deepPurple,
        body: SlotMachine(),
      ), // Scaffold
    ), // MaterialApp
  );
}

```

Методы **SoundService** теперь асинхронные (`Future<void>`), поэтому добавьте **await** перед вызовами. Остальной код трогать не нужно:

```
Future<void> _spin() async {  
  if (_coins <= 0 || _isSpinning) return;  
  ✓ await SoundService.playClick(); // ← добавьте await
```

Обновите внутри метода **\_spin()** блок кода который идёт вместе с **Future.delayed**:

```
// Короткая пауза перед результатом — финальная интрига  
await Future.delayed(Duration(milliseconds: 300));  
// Сначала определяем результат  
String newMessage;  
int coinsChange;  
if (result1 == result2 && result2 == result3) {  
  if (result1 == 'assets/images/seven.png') {  
    coinsChange = 10;  
    newMessage = 'ДЖЕКПОТ! 🎰🎰🎰 +10 монет';  
    await SoundService.playJackpot();  
  } else {  
    coinsChange = 3;  
    newMessage = 'Победа! 🎉 +3 монеты';  
    await SoundService.playWin();  
  }  
} else {  
  coinsChange = -1;  
  newMessage = 'Попробуй ещё раз 😞 -1 монета';  
  await SoundService.playLose();  
}  
// Потом обновляем UI — обычный синхронный setState  
setState(() {  
  _isSpinning = false;  
  _coins += coinsChange;  
  _message = newMessage;  
});
```

## Шаг 6. Проверяем на обеих платформах

**Web:**

**flutter run -d chrome**

**Android:**

Запустите эмулятор из Android Studio, затем выполните:

**flutter devices**

Убедитесь что эмулятор появился в списке:

```
➤ $ flutter devices  
Found 3 connected devices:  
+ sdk gphone64 x86 64 (mobile) • emulator-5554 • android-x64 • Android 16 (API 36) (emulator)  
Linux (desktop) • linux • linux-x64 • CachoOS 6.19.3-2-cachyos  
Chrome (web) • chrome • web-javascript • Google Chrome 145.0.7632.116
```



Запустите приложение:

**flutter run -d emulator-5554**

Звук должен работать на обеих платформах. Но на эмуляторе есть один баг со звуком, который связан с тем, что любой новый звук это потенциальный захват аудиофокуса. А Web этого не делает — поэтому разница поведения. В рамках данной работы мы не будем исправлять эту ошибку.

Единственное для улучшения работы мы можем убрать дублирование в `slot_machine.dart` — `playBackground()` теперь вызывается только из `initState`, флаг `_backgroundStarted` уже внутри сервиса:

```
// Убери поле:  
//var _backgroundStarted = false;
```

```
// В методе _spin() убери блок:  
//if (!_backgroundStarted) {  
//   SoundService.playBackground();  
//   _backgroundStarted = true;  
//}
```

Выполните в терминале:

**git add .**

**git commit -m "Update SoundService to use audioplayers for Web and Android"**

### Шаг 7. Собираем APK

Выполните в терминале:

**flutter build apk --release**

Первая сборка займёт **3–7 минут** — Android Gradle скачивает зависимости. Последующие сборки будут быстрее.

```
└─>$ flutter build apk --release  
Font asset "MaterialIcons-Regular.otf" was tree-shaken, reducing it from  
ction). Tree-shaking can be disabled by providing the --no-tree-shake-ico  
Running Gradle task 'assembleRelease'... 34,0s  
✓ Built build/app/outputs/flutter-apk/app-release.apk (54.0MB)
```

Готовый APK появится по пути:

**build/app/outputs/flutter-apk/app-release.apk**

 **СКРИНШОТ 3:** Скриншот терминала с успешной сборкой APK. Сохраните в **img/step3\_ВашаФамилия.png**



Выполните в терминале:

**git add .**

**git commit -m "Add Android release build"**

## Часть 6. Оформляем репозиторий на GitHub

### Шаг 1. Что такое хороший публичный репозиторий

Ваш GitHub — это портфолио. Работодатели и коллеги смотрят на репозитории когда оценивают разработчика. Хороший репозиторий отличается от плохого прежде всего **README.md** — это первое что видит любой человек зашедший на страницу проекта.

### Шаг 2. Добавляем скриншоты в репозиторий

Для **README** нам нужны скриншоты приложения прямо в репозитории. Создайте папку **steps/** и поместите туда несколько скриншотов готового приложения:

**mkdir steps**

Сделайте скриншоты запущенного приложения и сохраните в **steps/**:

**steps/**

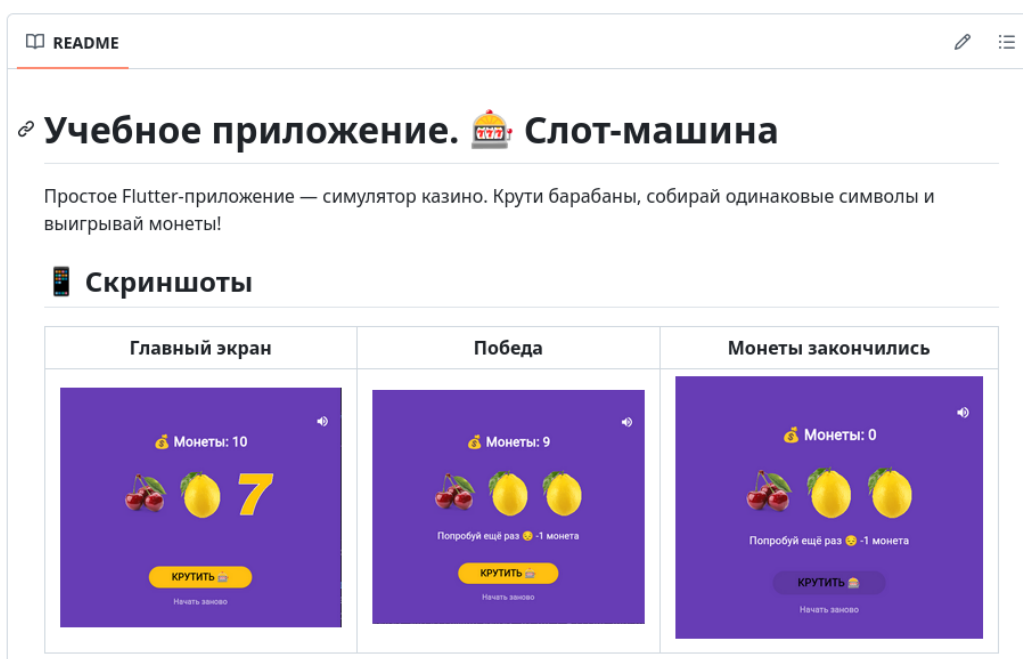
|— **main\_screen.png** ← главный экран

|— **win.png** ← экран победы

|— **no\_coins.png** ← экран без монет

### Шаг 3. Пишем README.md

Откройте **README.md** в корне проекта и напишите полноценное описание. Вот структура и пример для слот-машины:



## Как играть

- Нажмите **КРУТИТЬ** чтобы запустить барабаны
- Три одинаковых символа — победа (+3 монеты)
- Три семёрки — джекпот (+10 монет)
- Разные символы — проигрыш (-1 монета)
- Начните заново кнопкой **Начать заново**

## Запуск проекта

**Требования:** Flutter 3.x, Dart 3.x

```
# Клонировать репозиторий
git clone https://github.com/Paltosik92/slot_machine

# Перейти в папку
cd slot_machine

# Установить зависимости
flutter pub get

# Запустить в Chrome
flutter run -d chrome
```



## Установка на Android

Скачайте готовый APK:

[app-release.apk](#)

## Технологии

- Flutter 3.41.2
- Dart 3.11.0
- Платформы: Web, Android

## Что изучено

- `StatefulWidget` и управление состоянием через `setState()`
- Работа с локальными изображениями через `Image.asset()`
- Генерация случайных чисел через `dart:math`
- Анимация через `async/await` и `AnimatedOpacity`
- Создание иконки в Krita и подключение через `flutter_launcher_icons`
- Сборка под Web и Android

## Автор

Иванов Иван — группа ИСП-234  
Лабораторная работа №6, 2026

## Шаг 4. Загружаем на GitHub

Выполните в терминале:

**git add .**

**git commit -m "Final: add steps and polish README"**

**git push**

Откройте репозиторий на GitHub и убедитесь что:

- README отображается на главной странице
- Скриншоты видны в таблице
- APK доступен по ссылке



**СКРИНШОТ 4:** Скриншот страницы репозитория на GitHub с оформленным README и скриншотами. Сохраните в **img/step4\_ВашаФамилия.png**

**Выполните в терминале:**

**git add .**

**git commit -m "added README description and completed labs"**

**git push**